Future Generation Computer Systems 144 (2023) 24–36

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

FedDCS: A distributed client selection framework for cross device federated learning

Monalisa Panigrahi *, Sourabh Bharti, Arun Sharma

Indira Gandhi Delhi Technical University for Women, Delhi, India

ARTICLE INFO

Article history:
Received 24 May 2022
Received in revised form 17 December 2022
Accepted 2 February 2023
Available online 10 February 2023

Keywords:
Federated learning (FL)
Client selection
Internet of Things (IoT)
Resource aware selection
Distributed deep learning

ABSTRACT

In cross-device Federated Learning (FL) the existing client selection approaches are centralized, making the assumption that the participatory clients are resource-full and restrict their type to mobile phones. These assumptions limit the applications of FL and make it difficult for the global server to monitor the client (s) resources; as a consequence of which a high client drop-out rate is observed. Thus, a distributed cross-device FL framework with the provision of resource & time-aware client selection is required. This paper proposes a distributed cross-device FL client selection framework (FedDCS) which employs a novel data-type, resource, and time-aware protocol for FL client selection. FedDCS is designed to offload the client selection and their resource monitoring processes to the local edge servers with the objective to minimize client(s) drop-outs, model convergence time, number of message exchanges, and time to achieve a certain level of accuracy. Extensive experiments on two benchmark data sets: MNIST and FashionMNIST, are performed on the PyTorch-based FL platform to evaluate the performance of FedDCS on such metrics. The results show 100% client retention in FedDCS during FL model training for both MNIST and FashionMNIST datasets using IID and Non-IID settings. For the MNIST dataset, FedDCS shows 82% and 36% improvement in the time taken to reach the desired testing accuracy, and 18% and 10% improvement in model convergence time, for IID and Non-IID setting respectively as compared to the best performing state-of-the-art, i.e., FedMCCS. For the FashionMNIST dataset, FedDCS shows 17% and 6% improvement in the time taken to reach the desired testing accuracy, and 4% and 1% improvement in model convergence time, for IID and Non-IID setting respectively as compared to the best performing state-of-the-art, i.e., FedMCCS. In addition, FedDCS also significantly reduces the number of message exchanges during FL.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

Federated Learning (FL) is a distributed learning technique that allows IoT edge devices to train a machine learning (ML) model in a collaborative fashion, without having to share their raw data with each other. A typical training round of cross-device FL set-up [1] is executed with a large number ($<$100,000) of IoT edge devices (FLClients) in which a global model is trained using their local data and the model updates are sent to a centralized server for aggregation. This is followed by a centralized server updating the global model with the aggregated parameters and distributing the newly created global model to the FLClients for the next round of local training. This process iterates till the global model converges. Such collaborative model training not only provides FLClients access to a variety of training data-sets but also preserves their local data privacy and minimizes the amount of data exchange between data-set owners and the centralized server. In a cross-device FL set-up, the quality of the federated global model depends heavily upon the type of training data, FLClients' availability to train the global model, and network resource availability to exchange model updates with centralized server [2,3]. Thus, application-specific (i.e., in possession of relevant training data) and reliable FLClient selection is of paramount importance before the commencement of FL training. The type of FLClients in classical implementations of FL is limited to mobile phones [4], training the global model for a limited number of applications utilizing common data-types (i.e., text and images).

Issues with Current Frameworks: Current FLClient selection frameworks borrow the type of FLClients from such classical implementations. The client selection approaches in existing frameworks [2–4] are based on the assumption that FLClients are to participate in the FL process during their idle hours (i.e., at night) when no other applications are using their computational and communication resources. However, due to growing digitization and agile applications in need of a variety of training data, FLClients are expected to react and respond at any time, irrespective of their resource-availability. Moreover, existing client selection frameworks are agnostic to the data-types of

* Corresponding author.
E-mail address: monalisapanigrahi@igdtuw.ac.in (M. Panigrahi).

https://doi.org/10.1016/j.future.2023.02.001
0167-739X/© 2023 Elsevier B.V. All rights reserved.

such distributed FLClients, whereas recent FL implementations in domains such as smart manufacturing [5] are expected to utilize a variety of data-types to fully unlock the potential of privacy-preserving, collaborative model training.

In addition to the above-mentioned issues, the centralized server in these frameworks is responsible for both model aggregation and coordination of the client selection process. This centralized FLClient selection approach results in an exhausted server being a single point of contact for all the clients, resulting in loss of information in case of server failure. In addition, end-to-end client–server communication is not always reliable due to limited communication resources with FLClients. This can hamper the accuracy of the global model as FLClients' updates may get lost during model exchange [2,3].

Example Scenario: This paper considers a dynamic collaborative model training scenario which expects FLClients with the required training data-type to reliably support FL operations. This requires (a) agile discovery of (b) resource-full FLClients who can perform the model training and exchange without dropping out and with (c) the minimum number of message exchanges between them and the global server. One such application of this can be smart manufacturing where multiple manufacturing sites are willing to share their intelligence with each other in a privacy-preserving manner for mutual benefit. The use-case of collaborative *predictive maintenance* involves various IoT edge devices monitoring the machine-level parameters such as temperature, vibration etc. to predict its remaining use-full lifetime (RUL). Similar machines working under different operating conditions at different sites can exchange their local models with each other to learn unseen patterns. Being a time-critical use-case, it requires agile discovery of reliable FLClients in possession of different data-types required by the use-case.

Difference from Existing Solutions: The random client selection approach used in [4] observes client drop-out resulting in increased model convergence time, whereas the resource-based approach [2] is able to complete the training before a deadline but accuracy is not guaranteed because of more client drop-out. However, [3] minimizes client drop-out by selecting the resource-full clients, but the communication between client and server increases exponentially as the client shares their resource information with the server in each FL round. In addition to this, most of the resource-based FL client selection approaches share some amount of clients' resource information with the global server, which violates the primary motivation of FL, i.e., preserving clients' data privacy. A hierarchical federated learning (HFL) system known as HierFAVG [6] uses multiple edge servers between the clients and global server to reduce the model training time and energy consumption of clients by performing partial model aggregation at edge servers, however, the client selection in HierFAVG is random and performed by the global server in a centralized fashion.

The proposed FedDCS framework differs from the centralized FL framework [2–4] primarily by employing the local gateway nodes into the FL framework for FLClient selection. The proposed distributed framework is able to reduce the burden on the global server by offloading the task of FLClient selection to the local edge gateways. The consideration of actual resources of clients before selection makes FedDCS more robust against client drop-outs as compared to state-of-the-art. Unlike [2–4], FedDCS does not allow the clients to communicate directly with the global server resulting in a significant decrease in message exchanges, as shown in Section 6.4. Additionally, FedDCS does not compromise the clients' privacy by sharing any resource information with the global server. The proposed FedDCS framework is built over HFL to reduce communication overhead and model convergence time. The role of gateways in traditional HFL is limited to partial model aggregation whereas the proposed FedDCS employs local gateways for resource-aware client selection in order to reduce client dropout during model training and improve global model performance. In addition, FedDCS provisions to avoid the global server's interaction with irrelevant gateways in each FL round by selecting the gateways with the required data-type. Client heterogeneity and their available computation and communication resources are ignored in HFL as the objective is hierarchical model aggregation, on the other hand, the objective of FedDCS is to enable resource-aware FLClient selection to minimize the client drop-out, and achieve increased global model accuracy.

Research Contributions: Scenarios such as those discussed above involve a huge number of IoT edge devices (i.e., FLClients) and thus it can become cumbersome for the centralized server to discover suitable and reliable FLClients as it requires constant resource and training data-type monitoring. Centralized management can not only exhaust the server but also introduce delays in FLClient discovery. To this end, this paper makes the research contributions as follows:

1. A novel distributed client selection framework (FedDCS) for cross-device FL with the objective to reduce the client drop-outs, message exchanges, model convergence time and time to achieve a certain level of accuracy is proposed.
2. A mechanism supporting the agile discovery of FLClients by the global server. This is enabled by creating virtual groups of geo-distributed homogeneous clients over time. This further reduces the client selection time subsequently resulting in faster global model convergence.
3. A resource & time-aware client selection mechanism that significantly reduces the FLClient drop-out rate during model training. To make it privacy-aware, FedDCS is executed on local gateways (instead of a global server) where they can easily monitor and estimate a client's available resources and time to complete the received global model training, respectively.
4. An experimental performance analysis of FedDCS against existing client selection mechanisms. They are evaluated with training of deep neural networks, where clients' data were generated using two publicly-available benchmark image data-sets as MNIST and FashionMNIST. The experimental results reveal that FedDCS outperforms the state-of-the-art in terms of client drop-out, training time, model convergence time, message exchanges etc.

The rest of this paper is laid out as follows. Section 2 discusses the background and related research, while Section 3 discusses the system components and problem definition. In Section 4, proposed FedDCS client selection framework is described, while Section 5 describes the evaluation methodology. Section 6 explains the comparative analysis of results and complexity, followed by the conclusion and future work in Section 7.

2. Background and related research

A typical cross-device FL set-up (Fig. 1) generally consists of multiple FLClients training a global model in a collaborative fashion without revealing their raw data to each other. The FL process is orchestrated by a centralized server initializing the global model parameters based on the application requirements. As shown in Fig. 1, the following key steps are involved in executing an FL process.

Step 1. *Client Selection and Global Model Distribution:* The global server randomly selects a fixed number of clients from a set of available clients for training in each round and distributes the initial global model (w_t) along with its requirements to the selected clients (FLClients).

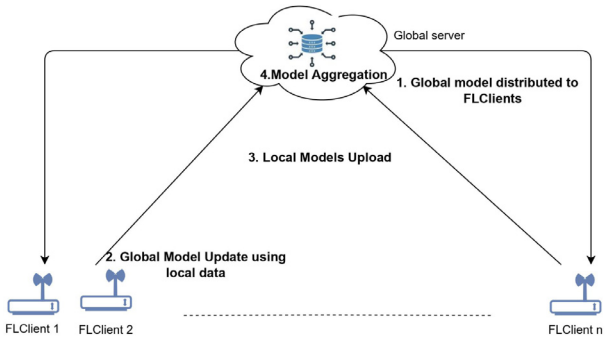


Fig. 1. Traditional federated learning architecture.

Step 2 & 3. *Update and Upload:* For every communication round (t), each FLClient (k) trains the received global model (w_t) with its local data, computes its local weight (w_t^k). Let, d_k = local training data set samples for client k, then, total size of data samples from K clients = $D = \sum_{k=1}^{K} d_k$. Federated learning aims to optimize a global loss function ($l(w)$) by minimizing the average local loss function ($l_k(w)$) of each client (k), i.e.,

$$\min_{w} l(w) = \sum_{k=1}^{K} \frac{d_k}{D} L_k(w) \tag{1}$$

where, $L_k(w) = \frac{1}{d_k} \sum_{i \epsilon d_k} l^i(w)$
Here, $l^i(w)$ = local loss function of data sample i in the data set of client k.
Every FLClient (k) computes its average gradient by using an optimization algorithm such as stochastic gradient descent (SGD) to minimize their loss function ($L_k(w)$) during local model training. After the completion of local training, every FLClient shares the updated global model (w_{t+1}^k) with the global server, which is expressed as:

$$w_{t+1}^k = w_t - \eta_k \Delta_k \tag{2}$$

where, η_k = learning rate of FLClient k and Δ_k = average gradient of FLClient k computed on its local dataset with the current global model (w_t)
Step 4. *Aggregation:* The global server collects the updated weights (w_{t+1}^k) from all the FLClients, performs aggregation and computes the updated global model (w_{t+1}) as follows:

$$w_{t+1} = \sum_{k=1}^{K} \frac{d_k}{D} w_{t+1}^k \tag{3}$$

The updated global model is again distributed to the FLClients for the next round and the same process continues until the convergence of global model.

2.1. Review on related research

To include the maximum number of relevant papers, a two-step selection criteria is followed. In step 1, the reputed digital portals such as IEEE, ACM, Springer, Elsevier are targeted to identify the appropriate journals and conferences for extracting the relevant papers. To carry out an effective survey all the concerning contributions starting from the inception of federated learning, i.e., from the year 2017 to October 2022 are studied. The keywords employed in search of related papers are federated learning, federated learning in IoT environment, distributed architecture in federated learning, edge computing, client selection, resource-aware client selection, data-type aware federated learning, etc. The detailed study of abstracts of the papers that implied the above-mentioned keywords is done to retrieve relevant papers. In step 2, the set of selected papers from step 1 is studied in detail and analysed further. Finally, specific contributions pertaining to the focus area of this work i.e., client selection in resource-constrained IoT environments are studied extensively to identify the research gap and comparative analysis.

The primary motivation of FL is to enable clients to train a global ML model in a collaborative fashion while preserving their raw data privacy. In this context, two critical factors affecting the ML model's accuracy are the number of clients participating in the training process and their training data. Thus, the selection of reliable clients (with enough computational resources) is crucial to minimize client drop-outs (Eq. (4)). Moreover, efficient client selection helps minimize the network resource consumption and improves the performance of the model in any distributed learning environment. Various research papers in the field of client selection in FL have been published for this purpose. While, most of the initial work inherits the traditional federated learning approach [4], the client selection approaches in cross-device FL fall into three broad categories discussed as follows.

(1) Random Selection: In random selection, the clients are selected randomly for training in each FL round. Google introduced the first federated learning algorithm, FedAVG [4], where the clients were selected randomly for local FL training by the global server followed by the aggregation of each client's update. This process is iterated for multiple rounds until the global model reaches the desired accuracy. A random client selection approach was proposed in [7] to improve the performance of non-iid data by sharing a small set of data across the clients. A Tier-based Federated Learning System, TiFL, was presented in [9] to alleviate the straggler problem caused by heterogeneity in terms of resources and data amount of clients, where the clients were first grouped into different tiers on the basis of their training accuracy and selected randomly from the same tier in each FL round. In all the existing random selection approaches, the overall training process can become inefficient in terms of the long update and upload time if some clients with limited computational resources are selected. These limitations of random client selection in federated learning motivate the research in the area of efficient client selection while considering the limited resources of clients.

(2) Resource-based Selection: In this type of selection, the global server considers the different available resources of clients while selecting clients for FL training. Various resource-based client selection approaches were introduced so far. FedCS, a federated learning client selection algorithm, proposed in [2] selected the clients for FL training, who could finish the training within a predefined deadline. FedCS's results produced a high-performance FL model in a fraction of the time it took to train using state-of-the-art FL protocol. FedCS mechanism is extended for non-iid data, known as hybrid learning technique in [10]. Another approach proposed in [11] derived the optimal number of edge devices to be selected as FLClients in order to minimize the average training time while guaranteeing convergence. An energy-aware client selection method, in which the client selection took into account the amount of data, communication quality, and residual battery power of clients, was proposed in [12]. A client selection approach (FedMCCS), in which clients were selected on the basis of their available resources (memory, time, energy) was proposed in [3]. Another client selection mechanism (OCEAN), in which currently available wireless channel information served as a criterion for client selection was proposed in [13] whereas, the client selection mechanism (CSFedAVG) proposed in [14] selects the clients with a lower degree of non-IID data for FL training. Another client selection and resource allocation mechanism where clients were selected based on their data distributions and communication

Table 1
Qualitative comparison of FedDCS with existing client selection mechanisms.

Mechanism	Framework	Resource aware	Data-type aware	Client drop-out	Client heterogeneity	Privacy preservation
[4] 2017	Centralized	No	No	Yes	No	Yes
[7] 2018	Centralized	No	No	Yes	No	No
[2] 2019	Centralized	Yes	No	Yes	Yes	No
[8] 2019	Centralized	Yes	No	Yes	Yes	Yes
[9] 2020	Centralized	No	No	Yes	Yes	Yes
[10] 2020	Centralized	Yes	No	Yes	Yes	No
[3] 2021	Centralized	Yes	No	Yes	Yes	No
FedDCS	**Distributed**	**Yes**	**Yes**	**No**	**Yes**	**Yes**

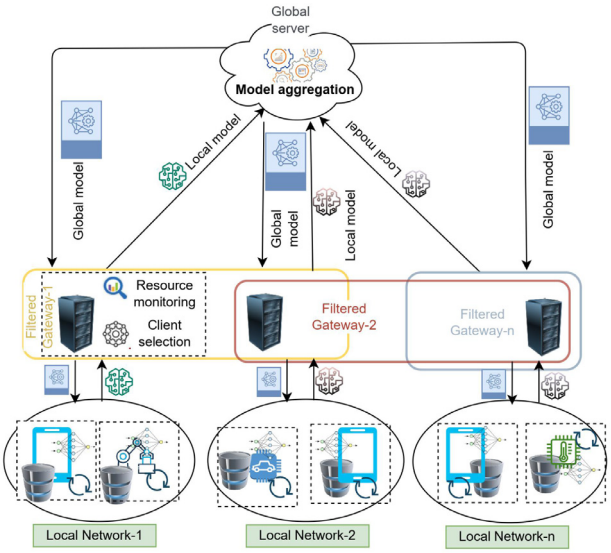


Fig. 2. Proposed federated distributed client selection (FedDCS) framework.

constraints for HFL framework was proposed in [15] to support remote monitoring of patients in intelligent healthcare services. HFL is combined with asynchronous FL in [16] to reduce the communication overhead between devices and the global server. A joint learning-based mechanism was proposed in [17] to optimize the number of local computations, edge aggregation, and user-to-edge association in HFL.

In most of the existing resource-based client selection approaches [2,3,10] the clients need to share their number of data samples as a piece of resource information to the server, breaching the privacy of clients, which is the primary motivation of federated learning. To mitigate this limitation, some incentive-based client selection approaches were introduced.

(3) Incentive-based Selection: In this approach, the global server introduces incentive mechanisms to motivate the clients to participate. The incentive mechanism can be derived from (i) the reputation of clients in terms of model convergence time, accuracy, etc., (ii) the stackelberg game, and (iii) the contract theory approach. A contract-based incentive system to encourage clients with finest data to actively participate in FL training was proposed in [8], whereas [18] proposed a novel incentive mechanism that combines reputation and contract theory to select reputed clients for FL training. A selective model aggregation strategy in which local DNN models were selected by evaluating the local image quality and computation capability was proposed in [19]. A reliable worker selection scheme based on reputation for federated learning tasks was proposed in [20], whereas a client selection approach based on reputation scores was proposed in [21] for the FL environment. An incentive-based mechanism using the stackelberg game was proposed in [22–24] to derive device participation in the federated learning process.

Apart from these categories, the approach proposed in [25] addresses the problem of client selection in HFL by considering the statistical properties of local datasets i.e., data distribution of clients and communication constraints. The client selection criteria is the similarity in their data distribution with the objective to increase the accuracy (in the non-iid scenarios) and minimize the delay. In contrast, the data-type in FedDCS is specified by the application as a required training data-type (Section 3.1). For instance, in IoT-enabled predictive maintenance applications in a manufacturing scenario, multiple edge devices may possess a different type of training data pertaining to different parameters such as vibration, temperature etc (Section 1). Here, the relevant training data selection is important for accurate global model training. Such data-types in FedDCS are used to create the virtual groups of the local gateways having at-least one client in their local networks with the data-type specified in the global model training task and are not used as a criterion for FLClient selection as such. In addition, unlike the solution proposed by [25] the proposed FedDCS is resource-aware and achieves not only high accuracy in the minimum FL rounds but also with minimum client drop-outs and message exchanges. Also, the tasks of client selection and their resource monitoring are delegated to the local gateways (edge servers) to make the framework scalable in the event of a large number of FLClients.

Synthesis: From the above discussion, it is clear that all the existing client selection approaches involve a global server as a central entity for client selection, which results in an exhausted server being a single point of contact for all the clients. The random selection approaches are unable to handle client drop-out which leads to increased model convergence time whereas the client's privacy is compromised in resource-based approach. Furthermore, there are limited research works on client selection for FL applications using resource-constrained IoT devices.[1] Therefore, a distributed resource-aware client selection framework, FedDCS is proposed to handle client drop-out while monitoring each client's resources without overburdening the server and compromising clients' privacy. The comparison of FedDCS with the existing client selection mechanisms is summarized in Table 1.

3. System components and problem definition

FLClients are resource-constrained IoT devices in possession of local data-sets corresponding to their data-types. It is assumed that each client is capable of providing unique data-types[2] throughout the FL process. Each client belongs to a local network, managed by the corresponding local gateway (Fig. 2). Each local gateway serves as an interface between the global server and its corresponding clients during the global model training process. The system consists of the following key entities:

[1] The terms 'IoT device' and 'Client' are used interchangeably throughout the paper.

[2] Data-type is referred to as the application-specific training data required to train the global model.

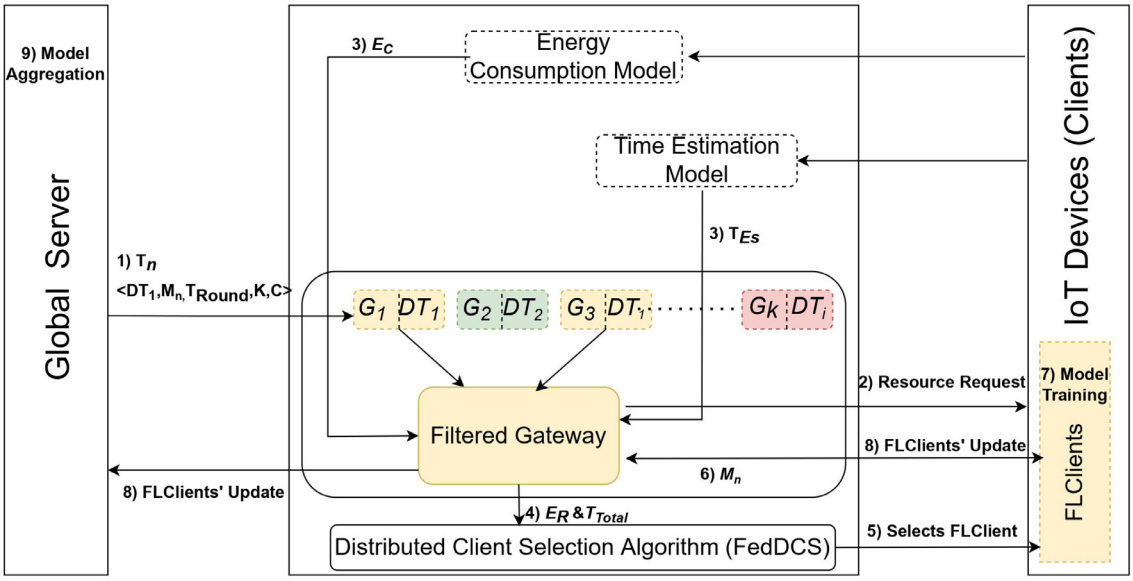


Fig. 3. Federated model training using FedDCS.

- *Global Server*: A unique global server acts as a centralized orchestrator for the FL process. It is responsible for generating the initial training task, selecting and coordinating with filtered-gateways (constituting IoT devices with the required data-types) and aggregating all the client updates received from their corresponding local gateways.
- *Local Gateway*: Each local network consists of multiple IoT devices connected to a unique local gateway acting as an interface between the global server and local IoT devices (clients). The local gateway is in possession of the initial energy level information of IoT devices in the local network. The responsibility of the local gateway is to collect resource information from IoT devices in the network and select appropriate FLClients in each FL round. This is followed by receiving the trained model parameters from FLClients and sharing them with the global server for aggregation.
- *IoT Devices*: IoT devices are responsible for sensing and gathering data corresponding to a particular data-type. These devices can communicate with their local gateways in a single-hop manner. Once selected as FLClients, IoT devices can perform FL training on their local data-sets using their onboard computational resources and communicate the updated model to the corresponding local gateway.

3.1. Problem definition

For an FL round n, the global server creates the model training task T_n, defined as a 5-tuple $\langle RDT_n, M_n, T_{Round}, K, C\rangle$, where RDT_n is the required data-type, M_n is the global model architecture, T_{Round} is the deadline to complete one FL round. K (total number of available clients for FL training) and C (fraction of clients to be selected for FL training) are the hyper-parameters provided by the global server. Let, $DT = \{DT_1, DT_2, \ldots, DT_i\}$ represents a set of i data-types, out of which the global server can request a maximum of one required data-type (RDT_n) per FL process. Let $G = \{G_1, G_2, G_3, \ldots, G_k\}$ be the set of k local gateways, where each local gateway is connected to the global server and provides a particular data-type from DT. A local gateway is said to provide a particular data-type, if at least one of the IoT devices in its local network is in possession of the training data corresponding to that data-type. Each IoT device can provide a maximum of one data-type throughout the FL process. For each FL round, the global server selects a maximum of $\lceil K \times C \rceil$ clients (IoT devices) from the local networks represented by their corresponding local gateways. Let, $C_S^n = \{C_{k1}, C_{k2}, C_{k3}, \ldots..C_{km}\}$ where $C_S^n \subseteq K$ be the set of selected clients for FL training round n. Similarly, if $C_d^n \subseteq C_S^n$ represents the set of clients which dropped out in between FL training round, the problem of minimizing the client drop-out addressed by FedDCS can be defined as:

$$\min \sum_{n=1}^{N} C_d^n \tag{4}$$

s.t.

$$|C_S^n| \leq \lceil K \times C \rceil \tag{5}$$

$$\forall C_{km} \in C_S^n, E_R^{C_{km}} \geq E_C^{C_{km}} \tag{6}$$

$$\forall C_{km} \in C_S^n, T_{Total}^{C_{km}} \leq T_{Round} \tag{7}$$

where, $E_R^{C_{km}}$ and $E_C^{C_{km}}$ represent the residual and consumed energy, respectively for client C_{km}. $T_{Total}^{C_{km}}$ is the estimated total time for client C_{km} and T_{Round} represents the deadline to complete one FL round.

4. FedDCS : FLClient selection framework

This section discusses the proposed FedDCS framework and explains the different steps involved in FLClient selection. As shown in Fig. 2, FedDCS envisages a distributed FL scenario where FLClients are part of their corresponding local networks which are located at different geographical sites. Such local networks are connected to the global server through local edge gateways which act as a key enabler for distributed FLClient selection in FedDCS framework. The overall process of federated model training using FedDCS is shown in Fig. 3. All the important key notations used to describe the FedDCS framework are listed in Table 2. FedDCS employs three (Algorithm 1, Algorithm 2 and Algorithm 3) sequentially to select the FLClients during training. Algorithm 1 is used to find out the set of relevant gateways for a particular task (T_n). Once the relevant (filtered) gateways are identified, then Algorithm 2 is used to select appropriate clients with enough resources to last the FL training. Using Algorithm 3,

Table 2
Key notations.

Notations	Description
N	Total number of FL rounds (0,1,2, …,n)
RDT_n	Required data-type by global server for FL round n
T_{Round}	Deadline to complete one FL round
K	Total number of available clients for FL training
C	Fraction of clients to be selected for FL training
G	Set of k local gateways
G_f^n	Set of i filtered gateways
G_{fi}^n	ith filtered gateway corresponding to ith data-type
G_{ik}^n	kth local gateway in ith filtered gateway(G_{fi}^n) providing data-type DT_i
$C^{G_{fi}^n}$	Set of clients inside a filtered gateway G_{fi}^n
$C^{G_{ik}^n}$	Set of m clients providing data-type DT_i corresponding to the local gateway G_{ik}^n
C_{km}	mth client from $C^{G_{ik}^n}$
$C_S^{G_{fi}^n}$	Set of selected clients inside a filtered gateway G_{fi}^n
$C_S^{G_{ik}^n}$	Set of selected clients by local gateway G_{ik}^n
$C_d^{G_{fi}^n}$	Set of clients from $C_S^{G_{fi}^n}$ which dropped out in between FL training round
$E_C^{C_{km}}$	Total consumed energy by an IoT device C_{km}
$E_R^{C_{km}}$	Residual energy of an IoT device C_{km}
$E_I^{C_{km}}$	Initial energy level of an IoT device C_{km}
$E_{comp}^{C_{km}}$	Energy consumption by an IoT device C_{km} in training local data sample for one local iteration/Epoch
$E_{comm}^{C_{km}}$	Energy consumption by an IoT device C_{km} for transmission of local update per round
$T_{Es}^{C_{km}}$	Estimated training time of an IoT device C_{km} to complete one FL round
$T_{comp}^{C_{km}}$	Required computational time by an IoT device C_{km} for one FL round
$T_{comm}^{C_{km}}$	Required upload time by an IoT device C_{km} per FL round
$T_{TTD}^{C_{km}}$	Time required to distribute the global model to the IoT devices by the local gateway
$T_{Total}^{C_{km}}$	Estimated total time of an IoT device C_{km} to complete one FL round
DR	Transmission rate of an IoT device
$\|w(M)\|$	Number of global model parameters
$T_{Total}^{Max}(G_{ik}^n)$	Maximum T_{Total} of each local gateway G_{ik}^n from the filtered gateway G_{fi}^n

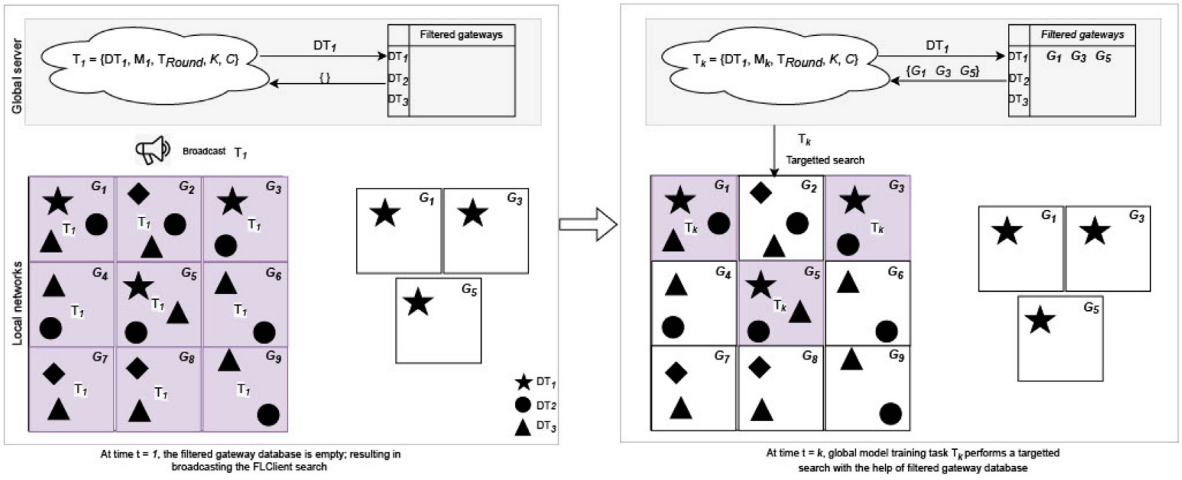


Fig. 4. Local gateway selection in FedDCS.

the global server selects the required number i.e., $\lceil K \times C \rceil$ clients (FLClients) from the set of clients selected by Algorithm 2 and shares the global model with them through their respective local gateways.

4.1. Efficient local gateway selection

The global server initiates the federated model training by creating a global model training task $T_n = \langle RDT_n, M_n, T_{Round}, K, C \rangle$ (Section 3.1) for each round of federated model training. As soon as T_n is created, it is important for the global server to search for FLClients in possession of the required training data-type (RDT_n). Since existing FLClient selection approaches such as those proposed in [2,4] are not data-type-aware, the task information is often broadcasted to the local networks for each round of federated model training, as shown in Fig. 4. This leads to the selection of irrelevant clients, increased/undesirable message exchanges resulting from broadcasting the task information for every round, and increased model convergence time which affect the global model performance. Another FLClient selection approach pro-

posed in [3] selects a smaller filtered set of clients based on their location, which decreases the number of message exchanges as compared to the state-of-the-art. However, the client filtering is done for each FL round which results in delayed FLClient selection and increased global model training time. To overcome the challenges faced in [2–4], FedDCS utilizes both filtering and storing information about the local gateways as shown in Fig. 4. To enable this, the global server applies stratified sampling [26] to filter the local gateways based on the required data-types. Stratified sampling classifies the gateways into subgroups known as strata which can be defined as a group of gateways sharing similar characteristics together. In this proposal, the global server stores the information about local gateways to *filtered-gateways* database if they provide the required data-types, i.e., if the gateway has at least one client in its network with the required data-types.

For T_n, global server filters a subset of gateways (known as *filtered-gateways*), $G_f^n = \{G_{f1}^n, G_{f2}^n, G_{f3}^n, \ldots, G_{fi}^n\}$ based on their provided data-types, where each filtered-gateway (G_{fi}^n) provides a single data-type DT_i such that, $|G_f^n| = i$. Each filtered gateway (G_{fi}^n) can consist of more than one local gateway such that $G_{fi}^n \subseteq G$. *i.e.*, $G_{fi}^n = \{G_{i1}^n, G_{i2}^n, G_{i3}^n, \ldots, G_{ij}^n\}$, where G_{ik}^n is the kth local gateway in G_{fi}^n, providing data-type DT_i. The set of clients inside a filtered gateway, G_{fi}^n can be calculated as:

$$C^{G_{fi}^n} = \bigcup_{\substack{k=1 \\ i}}^{j} C^{G_{ik}^n} \tag{8}$$

where, $C^{G_{ik}^n}$ is the set of m clients providing data-type DT_i corresponding to the local gateway $G_{ik}^n = \{C_{k1}, C_{k2}, C_{k3}, \ldots, C_{km}\}$ where, $C_{km} = m$th client in $C^{G_{ik}^n}$.

Each local gateway from its corresponding *filtered-gateway* selects a maximum of $\lceil K \times C \rceil$ clients from its local network. The set of selected clients inside a filtered gateway G_{fi}^n can be calculated as:

$$C_S^{G_{fi}^n} = \bigcup_{\substack{k=1 \\ i}}^{j} C_S^{G_{ik}^n} \tag{9}$$

where, $C_S^{G_{fi}^n} \subseteq C^{G_{fi}^n}$ and $C_S^{G_{ik}^n}$ be the set of selected clients by local gateway G_{ik}^n such that $C_S^{G_{ik}^n} \subseteq C^{G_{ik}^n}$.

Algorithm 1 is executed on a global server where *filtered-gateways* database is referenced for every global model training task. The stored information about local gateways is returned straightaway if the required data-type was requested before, otherwise, the global task information is broadcasted to every local gateway. information about local gateways is returned straightaway if the required data-type was requested before, otherwise, the global task information is broadcasted to every local gateway.

As shown in Fig. 4, the information associated with T_1 is broadcasted to every local network which results in $G_{f1} = \{G_1, G_3, G_5\}$ as these gateways manage clients which are in possession of DT_1. The information about G_{f1} is stored at the global server as to when another task T_k with the same data-type (DT_1) requirement is created, G_1, G_3 and G_5 are directly referenced without the need of performing another FLClient search (Fig. 4). This helps to avoid delays in FLClient selection and minimizes the message exchanges.

Algorithm 1 Local Gateway Selection Algorithm

Input : Global model training tasks (T_n)
Output: Filtered gateway (G_{fi}^n)

```
Initialization: Filtered gateway (G_fi^n) = ∅
for every task T_n do
    if (RDT_n == DT_i) then
        if (G_fi^n == ∅) then
            for every G_k ∈ G do
                if ∃m such that (DT(C_km) == RDT_n) then
                    G_fi^n = G_fi^n ∪ G_k
                end
            end
        end
    end
    return G_fi^n
end
```

4.2. Resource-aware distributed FLClient selection

Apart from the required training data-type, it is important for the global server to select resource-full FLClients which can perform global model training task without dropping out in between the FL process. As per existing approaches [2,3], such resource-aware FLClient selection is performed by the global server, which may lead to the event of single-point-of-failure as the number of FLClient increases. To mitigate this problem, FedDCS employs a fully decentralized approach where the task of FLClient selection is delegated to the local gateways. For resource awareness, each local gateway requests resource information such as total consumed energy [27], estimated required training time from the clients providing the required data-types. The client which receives the resource request estimates the energy consumption (E_C) and training time (T_{Es}) for a given task using the following models.

4.2.1. Energy consumption model

The given energy consumption model provides the estimation of the consumed energy E_C by a client C_{km} (i.e., $E_C^{C_{km}}$) for one round of federated model training. This estimation is necessary to predict if the client has sufficient residual energy to complete the federated model training in a successful manner. As we assume that FLClients are wireless IoT devices (Section 3), the energy consumption model is adopted from [28] where the total energy consumed by an IoT device $E_C^{C_{km}}$ can be calculated as:

$$E_C^{C_{km}} = (\text{Epochs} \times E_{comp}^{C_{km}}) + E_{comm}^{C_{km}} \tag{10}$$

where, $E_{comp}^{C_{km}}$ is the energy consumption of each IoT device C_{km} in training local data sample for one local iteration/Epoch, which is given by

$$E_{comp}^{C_{km}} = \zeta * C_n * S_n * (f_n)^2 \tag{11}$$

where, ζ is the effective capacitance parameter of the computing chipset for the device, where C_n is the required number of CPU cycles to train one data sample, S_n is the number of training data samples available with the client, f_n is the CPU cycle frequency, and $E_{comm}^{C_{km}}$ is the energy consumption of each IoT device C_{km} for transmission of local update per round, which is given by

$$E_{comm}^{C_{km}} = (E_t + E_d \times r^p)B \tag{12}$$

where, E_t is energy/bit consumed by the transmitter electronics, and E_d is the energy consumed in the transmit op-amp (including op-amp inefficiencies). Both E_t and E_d are the properties of the transceiver used by the IoT device whereas r represents the transmission range of the IoT device and p is the power index for the channel path loss of the antenna. Usually, p depends upon the RF environment and is generally between 2 and 4 whereas B (bit rate of the radio) is a fixed parameter in our study.

4.2.2. Training time estimation model

The given training time estimation model provides the estimate to the time (T_{Es}) required by each client C_{km} (i.e., $T_{Es}^{C_{km}}$) to complete one round of FL training. This estimation is necessary to predict if the client can complete the FL rounds successfully before the deadline (T_{Round}). $T_{Es}^{C_{km}}$ is measured by the total time required by client C_{km} to train the global model ($T_{comp}^{C_{km}}$) and time taken to upload the updated model parameters to the local gateway ($T_{comm}^{C_{km}}$).

$$T_{Es}^{C_{km}} = T_{comp}^{C_{km}} + T_{comm}^{C_{km}} \tag{13}$$

$T_{comp}^{C_{km}}$ is formally measured as the computational time required by the IoT device for one round of global model training which is given by

$$T_{comp}^{C_{km}} = \frac{C_n \times S_n}{f_n}, T_{comm}^{C_{km}} = \frac{S_n}{DR} \tag{14}$$

where C_n is the required number of CPU cycles to train one data sample, S_n is the number of training data samples available with the client, f_n is CPU cycle frequency and DR represents the transmission rate of the client i.e., the number of bits that can be transmitted per second.

Algorithm 2 Resource Aware Client Selection Algorithm

Input : Set of clients ($C^{G_{fi}^n}$) for a required data-type RDT_n

Output: Set of selected clients $C_S^{G_{fi}^n}$

Initialization: The set of selected clients ($C_S^{G_{fi}^n}$) = $\emptyset$
if ($|C^{G_{fi}^n}| > 0$) **then**
  for *every* $C_{km} \epsilon\ C^{G_{fi}^n}$ **do**
    calculate $E_C^{C_{km}}$ and $E_R^{C_{km}}$ using eq.10 and 15 respectively
    if ($E_R^{C_{km}} \geq E_C^{C_{km}}$) **then**
      $C_S^{G_{fi}^n} = C_S^{G_{fi}^n} \cup C_{km}$
      if ($|C_S^{G_{fi}^n}| > 0$) **then**
        calculate $T_{Total}^{C_{km}}$ using eq.15
        if ($T_{Total}^{C_{km}} > T_{Round}$) **then**
          remove C_{km} from $C_S^{G_{fi}^n}$
        end
      end
    end
  end
  return $C_S^{G_{fi}^n}$
end

4.2.3. FLClient selection

The proposed distributed client selection (Algorithm 2) is executed on a filtered gateway (G_{fi}). For each FL round, clients' residual energy resources ($E_R^{C_{km}}$) and estimated total time ($T_{Total}^{C_{km}}$) are compared against $E_C^{C_{km}}$ and T_{Round}, respectively. As per task's requirements, each IoT device exchanges $E_C^{C_{km}}$ and $T_{Es}^{C_{km}}$ with its corresponding local gateway for each FL round. Based on these values, $E_R^{C_{km}}$ and $T_{Total}^{C_{km}}$ are estimated by the local gateway as:

$$\begin{aligned} E_R^{C_{km}} &= E_I^{C_{km}} - E_C^{C_{km}} \\ T_{Total}^{C_{km}} &= T_{TTD}^{C_{km}} + T_{Es}^{C_{km}} \end{aligned} \tag{15}$$

where $E_I^{C_{km}}$ represents the initial energy level of the IoT device, $T_{TTD}^{C_{km}} = \frac{|w(M)|}{DR}$ is the time required to distribute the global model to the clients by the local gateway and $|w(M)|$ is the number of global model parameters. Each local gateway inside the filtered-gateway executes Algorithm 2 to select maximum $\lceil K \times C \rceil$ clients in its local network that are capable of performing model training task within a certain deadline. Only the clients with $E_R^{C_{km}}$ value greater than the $E_C^{C_{km}}$ are added to the selected set. Finally, clients with $T_{Total}^{C_{km}}$ less than T_{Round} can be selected for federated model training. As shown in Algorithm 3, after receiving the response from each local gateway (G_{ik}^n) of G_{fi}^n, the global server arranges the local gateways (G_{ik}^n) in the increasing order of their $T_{Total}^{Max}(G_{ik}^n)$. This is followed by the global server sending the acknowledgement (CU_R) to the local gateways in a sequential manner until the summation of a number of selected clients reaches the value of $\lceil K \times C \rceil$. This is to prioritize the clients with minimum $T_{Total}^{C_{km}}$ to be selected as FLClients in order to minimize the global model convergence time.

Algorithm 3 FLClient Selection Algorithm

Input : $C_S^{G_{fi}^n}$, $T_{Total}^{C_{km}} \forall C_{km} \epsilon\ C_S^{G_{fi}^n}$

Output: Required Client Updates (CU_R)

Part 1 - Local gateway:
for *each local gateway* $G_{ik}^n \in G_{fi}^n$ **do**
  calculates $|C_S^{G_{ik}^n}|$ and $T_{Total}^{Max}(G_{ik}^n)$ using Eq. 16 and shares with global server.
end
Part 2 - Global server:
▷ arranges local gateways G_{ik}^n in the ascending order of their $T_{Total}^{Max}(G_{ik}^n)$ into a list Z.
▷ $CU_R \leftarrow \lceil K \times C \rceil$
for ($j = 0$ *to* $|Z|$) **do**
  if ($CU_R > 0$) **then**
    if ($CU_R < |C_S^{Z[j]}|$) **then**
      global server sends CU_R to $Z[j]$.
      $Z[j]$ forwards M_n to CU_R clients.
      exit
      else
        global server sends $|C_S^{Z[j]}|$ to $Z[j]$.
        $Z[j]$ forwards M_n to $|C_S^{Z[j]}|$ clients.
        $CU_R = CU_R - |C_S^{Z[j]}|$.
      end
    end
  end
  return CU_R
end

Upon receiving the acknowledgement from the global server, the local gateway forwards M_n to the FLClients which use their local data to update the M_n and upload the updated M_n to their respective local gateways. The detailed steps included in FL model training enabled by client selection using FedDCS are explained in protocol 1. Following the resource-aware client selection, each local gateway (G_{ik}^n) from the filtered gateway (G_{fi}^n) shares the total number of selected clients $|C_S^{G_{ik}^n}|$ in its local network and maximum T_{Total}, i.e., ($T_{Total}^{Max}(G_{ik}^n)$) with the global server, which can be calculated as:

$$\begin{aligned} T_{Total}^{Max}(G_{ik}^n) = Maximum(T_{Total}^{C_{km}}) \\ s.t.\ C_{km} \in C_S^{G_{ik}^n} \end{aligned} \tag{16}$$

After receiving the required clients' updates through the respective local gateway, the global server performs aggregation as in [4].

Table 3
Simulation parameters.

Parameter	Value
K	1000
C	0.1
$\|G\|$	10
N	100
Number of epochs per FL round	5
Learning rate	0.00001
Local batch size	5
Model optimizer	SGD
ζ	10^{-28}
C_n	10
f_n	1
T_{Round}	5 min
$E_l^{C_{km}}$	5000 J
E_t	50×10^{-9} J/bit
E_d	100×10^{-12} J/bit/m^2
r	30
p	2
B	1 Mb/s
DR	1–3 Mbps

5. Performance evaluation methodology

This section discusses the simulation environment, the training data set and its distribution over clients, the architecture of the global model, the baseline approaches and the evaluation criteria.

5.1. Simulation environment

FedDCS is simulated using Pysyft [29], where the clients/local gateways interact with each other as virtual workers using a system with an intel i5 processor, 1.19 GHz clock cycles and 8 GB RAM. A single global server, 10 local gateways, and 1000 clients/IoT devices (100 clients in each local network) are considered for the simulation environment. Each local network is assumed to contain the same number of clients, which are assumed to be resource-constrained IoT edge devices with different data processing capabilities. A data-type is also assumed to be associated with each client. Table 3 shows the key simulation parameters along with their values considered throughout the experiments.

5.2. About the dataset

In order to extensively test the robustness and efficiency of the proposed FedDCS framework, we used two different data-sets: MNIST [30] and FashionMNIST [31], which are publicly available and widely used in the literature [2,4,6,19,21]. MNIST [30] consists of handwritten digits with 60,000 training images and 10,000 testing images whereas, FashionMNIST [31] is a dataset of grayscale images, consisting of 60,000 training images and 10,000 testing images of 10 different fashion products such as T-shirts, bags, dress etc.

5.3. Data distribution over clients

In order to fully explore the utility of the proposed FedDCS in an FL set-up, the training dataset (MNIST/ FashionMNIST) is distributed among 1000 clients in two different settings: IID (Independent and Identically Distributed) and Non-IID. The dataset distribution is adopted from [2], where in IID setting every client is assigned an equal number of random images from the entire training dataset and in the Non-IID setting the training dataset is first partitioned into 2000 groups of 30 samples per group, then every client is randomly assigned samples from any 2 group [32]. In each round of FL, C is set to 0.1 and maximum $\lceil K \times C \rceil = 100$ clients can be selected. Finally, the testing dataset is used to measure the performance of the model.

Protocol 1: Federated model training protocol enabled by FedDCS. K represents the total number of available clients in the protocol. The fraction of clients chosen for FL training using FedDCS in each round is determined by the hyper parameter $C \in (0, 1]$.

1: *Initialization*: The global server creates a model training task (T_n).
2: *Efficient Local Gateway Selection and Model Distribution*: The global server applies stratified-based filtering to filter the local gateways based on the required data-types, avoiding communications with irrelevant gateways and distributes the initial model training task T_n to the selected filtered gateway.
3: *Resource-aware Distributed FLClient Selection*: Based on the clients' responses, The filtered-gateway uses the distributed client selection approach to select maximum $\lceil K \times C \rceil$ clients in its own network and shares the total number of selected clients and maximum training time required by the selected clients with the global server.
4: *Acknowledgement*: The global server sorts local gateways in the increasing order of their maximum training time and sends an acknowledgement to the local gateways sequentially until the summation of a number of clients in the local gateways reaches $\lceil K \times C \rceil$.
5: *Distribution*: After receiving the acknowledgement, the local gateways forward the initial global model (M_n) to their FLClients.
6: *Update and Upload*: FLClients use their local data to update the shared model and upload the new model parameters to the local gateway.
7: *Aggregation*: The global server receives the updates through the local gateway, and performs aggregation as in [4].
8: Steps 2 till 7 are iterated for multiple rounds until the model achieves the desired performance or the final deadline.

5.4. Global model architecture

The global model is a convolutional neural network (CNN) consisting of two 5×5 convolutional layers (one with 20 channels and the other with 50 channels). Then, each layer is activated by using ReLU activation function, followed by 2×2 max pooling and two fully connected layers [32].

5.5. Baseline approaches

The following baseline approaches are considered for comparative evaluation:

1. The original FL approach [4], where $\lceil K \times C \rceil$ clients are randomly selected by the global server to participate in the FL rounds. We refer to this approach as FedAVG.
2. FL with client selection [2], where $\lceil K \times C \rceil$ number of clients are randomly requested by the global server to provide their available resource information. Clients whose updates can reach to the global server before the set deadline are selected for FL training. We refer to this approach as FedCS.
3. FL with multicriteria client selection [3], where the clients are first filtered based on their data-types. This is followed by selecting $\lceil K \times C \rceil$ number of clients based on their available resources such as CPU, memory, energy, time etc. We refer to this approach as FedMCCS.

5.6. Evaluation criteria

The proposed FedDCS framework is evaluated in terms of the following metrics:

1. Number of Selected Clients Throughout the FL Process Without Dropping out: Since only a fixed fraction of clients (C) are selected (FLClients) out of available clients (K) for FL training in each round, it is important for the FLClients to complete the training task without dropping out in order to train a robust and accurate model.

2. Time Taken to Achieve the Desired Testing Accuracy: Testing is required to validate a trained model, but test time also plays an important role (faster is better) to avoid delays in achieving the desired accuracy of the test dataset. Delayed test time also increases the total time of getting an accurate model.

3. Model Convergence Time: The model convergence time determines the speed at which the model converges to the optimum loss. When the model converges at a minimum, a more accurate model is obtained in a lesser number of rounds.

4. Number of Message Exchanges Throughout the FL Process: Each communication round involves some message exchanges between the FLClient, the local gateway, and the global server. The more messages that are exchanged, the faster FLClient run out of resources and the higher the computing cost. Therefore, a minimum number of messages should be exchanged in each round while maintaining the quality of the model.

6. Results and analysis

This section analyses FedDCS's performance evaluation using evaluation metrics and compares the findings to baseline methodologies.

6.1. Number of selected clients throughout the FL process without dropping out

Fig. 5 depicts the number of selected clients able to complete the FL training using the proposed FedDCS along with baseline approaches when tested for MNIST (IID setting), MNIST (Non-IID setting), FashionMNIST (IID setting) and FashionMNIST (Non-IID setting) datasets, respectively. FedAVG is not resource-aware and follows a random selection of $\lceil K \times C \rceil$ number of clients from the available set of clients in each round. This results in 100% client selection initially, however, as the clients' resources are consumed over time, drop-out is observed in the later rounds. Unlike FedAVG, FedCS selects only those clients who can complete the training within a predefined deadline from a set of randomly selected $\lceil K \times C \rceil$ clients. However, FedCS does not consider the remaining energy level of the clients at the time of selection. Thus, the selected clients may drop-out during the FL process (Fig. 5). As compared to other approaches, FedCS observes the largest number of client drop-outs during the FL process as the clients in FedCS are selected from a smaller set of $\lceil K \times C \rceil$ clients instead of total available clients. This minimizes the chances of 100% client selection as shown in the result.

For FedMCCS, the $\lceil K \times C \rceil$ clients are selected from a set of filtered clients while considering different factors such as CPU, memory, energy, time, etc., of clients. Unlike FedAVG and FedCS, FedMCCS follows a linear regression model to predict the resource utilization of clients for the FL rounds based on the resource utilization in previous FL rounds. This approach is not suitable for the following scenarios: (1) If the number of clients is large, it becomes a tedious task for the global server to keep a record of the history of each client for the prediction, as the global server is already burdened with the task of utility prediction and client selection. (2) If the client's data distribution in any round is different from the distribution from which the history is collected; for an instance, if the utility prediction for clients with IID distribution is based on the history utility of clients with Non-IID distribution, then the predicted values can be less accurate. Thus, FedMCCS is a good choice for scenarios with a limited number of clients with fixed data distribution. On the other hand, one of the important challenges in FL remains to train the model using Non-IID distribution of clients [1]. Apart from this, both FedCS and FedMCCS demand the clients to share their private information such as sample size with the global server, which violates the primary motivation of federate learning, i.e., clients' privacy [4]. The proposed FedDCS framework is fully resource-aware and ensures that a client is selected for FL training only if it has sufficient resources to finish the training before a deadline. Unlike FedMCCS, FedDCS does not work on prediction based on history, instead it works on the actual values of clients' computation and communication resources resulting in 100% client availability during FL training. Also FedDCS does not share any private information of clients with the global server, retaining the primary objective of federated learning. The performance of FedDCS is also unaffected by a large number of clients (100 clients in each round) or different data distribution (IID/Non-IID), as shown in Fig. 5.

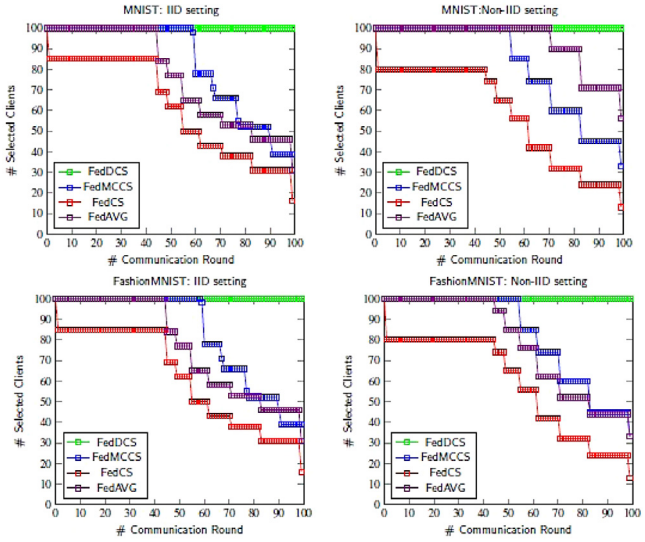


Fig. 5. Comparison of selected clients throughout the FL process for MNIST and FashionMNIST data-set using IID and Non-IID setting.

6.2. Time to achieve the desired testing accuracy

Tables 4–7 show the time taken by the trained model of each approach to reach a certain accuracy level for the test dataset for MNIST (IID setting), MNIST (Non-IID setting), FashionMNIST (IID setting) and FashionMNIST (Non-IID setting) respectively. The time to reach 50%, 60%, 70%, 80% and 90% of accuracy on testing dataset for MNIST (IID/Non-IID setting) and 40%, 50%, 60%, 70% and 72% of accuracy on testing dataset for FashionMNIST (IID/Non-IID setting) is observed in our experiment. Since FedCS does not have a sufficient number of clients for training in each round, it took the longest time, i.e., 13 min (MNIST/IID), 14 min (MNIST/Non-IID) to reach 80% accuracy respectively and 11 min (FashionMNIST/IID), 19 min (FashionMNIST/Non-IID) to reach 70% accuracy respectively as shown in Tables 4–7. Whereas, FedMCCS and FedAVG also take more time than FedDCS because these approaches also ran out of clients' data after a few rounds. As shown in the result, FedDCS is able to reach 80% accuracy and

Table 4
Comparison of required time (in *minutes*) to arrive at a testing accuracy.

MNIST-IID setting accuracy					
Approach	50%	60%	70%	80%	90%
FedDCS	1.66	2.43	3.74	5.78	**11.48**
FedMCCS	3.76	5.79	8.13	10.59	20.91
FedCS	4.11	5.73	9.46	13.16	NaN
FedAVG	2.15	3.73	6.99	10.24	26.21

Table 5
Comparison of required time (in *minutes*) to arrive at a testing accuracy.

MNIST-NonIID setting accuracy					
Approach	50%	60%	70%	80%	90%
FedDCS	4.53	5.46	6.51	8.08	**14.48**
FedMCCS	5.52	6.96	8.15	9.81	19.77
FedCS	8.29	8.94	10.26	13.99	NaN
FedAVG	9.02	10.18	11.11	14.02	NaN

Table 6
Comparison of required time (in *minutes*) to arrive at a testing accuracy.

FashionMNIST-IID setting accuracy					
Approach	40%	50%	60%	70%	72%
FedDCS	1.63	3.27	4.42	8.02	**10.56**
FedMCCS	2.50	4.82	6.20	10.44	12.32
FedCS	1.79	4.02	5.73	11.45	NaN
FedAVG	1.91	4.51	6.29	12.11	19.85

Table 7
Comparison of required time (in *minutes*) to arrive at a testing accuracy.

FashionMNIST-Non-IID setting accuracy					
Approach	40%	50%	60%	70%	72%
FedDCS	2.64	3.49	4.59	8.11	**13.45**
FedMCCS	2.22	4.03	4.88	8.99	14.27
FedCS	5.99	7.54	10.00	18.85	NaN
FedAVG	4.41	6.51	9.35	14.95	NaN

70% accuracy in as early as 8 min for MNIST and FashionMNIST respectively and which is earlier than the other approaches. The reason behind high accuracy in less time is that the model is trained more quickly than others, as it gets sufficient clients' data for training in each round.

6.3. Model convergence time

The result in Fig. 6 shows the number of communication rounds taken by an FL client to converge to an optimal loss using all the four approaches for MNIST (IID setting), MNIST (Non-IID setting), FashionMNIST (IID setting) and FashionMNIST (Non-IID setting) respectively. The loss value against each round is observed for all the approaches. The result clearly shows that FedDCS takes less time to converge and is very close to FedMCCS and FedAVG whereas FedCS takes a longer time to converge to an optimal loss value. The reason behind the slower convergence is the increased number of client drop-outs during training, whereas FedDCS has no client drop-outs resulting in faster convergence. The reduced convergence time helps the global server to get the trained model well before the final deadline.

6.4. Number of message exchanges throughout the FL process

Fig. 7 shows the total number of message exchanges between the global server and clients during the FL process using all four approaches for MNIST (IID setting), MNIST (Non-IID setting), FashionMNIST (IID setting) and FashionMNIST (Non-IID setting) datasets, respectively. The total number of message exchanges

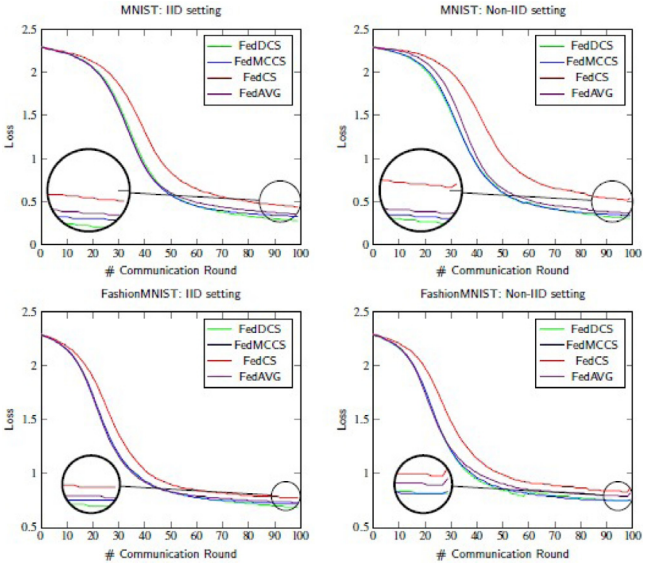


Fig. 6. Comparison of convergence time for MNIST and FashionMNIST data-set using IID and Non-IID setting.

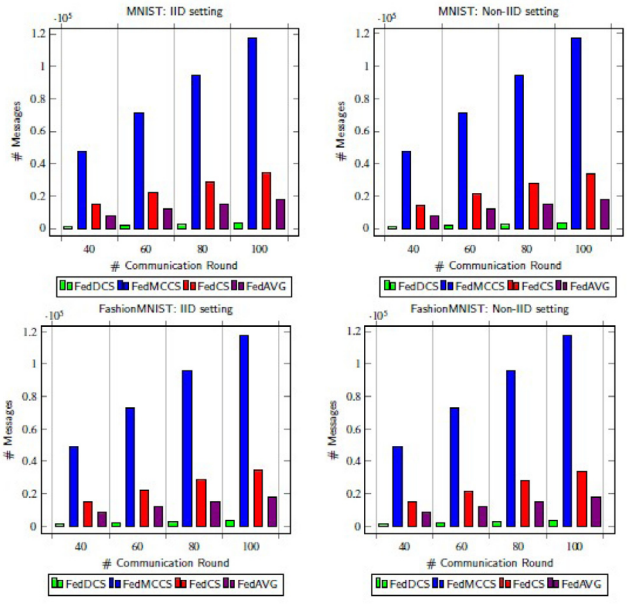


Fig. 7. Comparison of number of message exchanges for MNIST and FashionMNIST using IID and Non-IID setting.

in FedAVG includes the number of global model distribution messages from the global server to the selected clients and model update messages from the clients (who finish training before the deadline) to the global server. In FedCS, the number of message exchanges include the number of resource request and resource response message to/from $\lceil K \times C \rceil$ clients in addition to the number of global model distribution message to the selected clients and update message-only from the clients' who completed their training. In FedMCCS, the number of message exchanges include the number of resource request and resource response message to/from the filtered clients along with the number of global model distribution message to the selected clients and update message only from the clients who completed their training. The result (Fig. 7) shows that FedMCCS takes the maximum number of message exchanges because the number of filtered

clients is always high in number whereas FedCS sends/receives the resource messages to a maximum of $\lceil K \times C \rceil$ clients in each round and FedAVG does not send/receive any resource messages. However, as FedCS and FedAVG both involve the global server in client selection, it results in an increased number of message exchanges as compared to the FedDCS approach. The total number of message exchanges (data-types announcement messages, acknowledgement messages, global model distribution messages, client selection messages and update messages) in FedDCS is limited only to the communication between the global server and the local gateway. This is because of the distributed client selection in FedDCS as it allows only the local gateways to interact with the global server in each FL round.

Data-types announcement message and its acknowledgement are only sent in the next FL rounds if the required data-types in the current round are different. It is because FedDCS allows the global server to use the same filtered-gateways for the same data-types in the next FL rounds, which in turn reduces the number of messages. The fewer number of message exchanges in FedDCS eases the burden on the network by consuming the bandwidth wisely as shown in the result.

6.5. Complexity analysis

In FedAVG [4], maximum $\lceil K \times C \rceil$ clients are selected randomly from total available clients(K). Hence, the complexity of client selection in [4] is $O(1)$ per FL round. For each FL round, FedCS [2] selects K' number of clients randomly from total available clients(K) where, $|K'| \leq \lceil K \times C \rceil$ with a complexity of $O(|K'|)$ followed by selecting S number of resource-full clients where, $|S| \subseteq K'$ with a complexity of $O(|S|)$. Hence, the total complexity of client selection in [2] is $O(|K'| * |S|)$ per FL round. For each FL round, FedMCCS [3] first filters the clients (K_f) from total available clients(K) where $K_f \subseteq K$ and predicts the resources of filtered clients using historical data set of size n with the complexity of $O(n * |K_f|)$. Then, it employs a multi-criteria approach for client selection to select S clients, where $|S| \subseteq K_f$ and $|K_f| \leq \lceil K \times C \rceil$ with a complexity of $O(|S|)$. Hence, the total complexity of client selection in [3] is $O(n * |K_f| * |S|)$ per FL round.

For each FL round (n), FedDCS employs Algorithm 1 to filter local gateways providing a required data type (RDT_n) for a particular task (T_n). As shown in Algorithm 1, if T_n is requested for the first time (Step 4) we iteratively add the local gateways to the filtered gateway set (Steps 5, and 7) only if the clients in the local gateways provide the required data type i.e., RDT_n (Step 6). So, the complexity of this algorithm for the first FL round is $O(|G|)$, where G represents the set of local gateways. However, the complexity of this algorithm for the rest of the FL rounds is $O(1)$. Algorithm 2 is used to select resource-full clients from the set of clients inside the filtered gateway received from Algorithm 1. As shown in Algorithm 2, the clients with sufficient energy (Step 5) that can complete the FL training before the deadline (Step 9) are iteratively added to the selected client set (Steps 6, and 10). If $C^{G_{fi}^n}$ represents the clients inside the filtered gateway, the complexity of Algorithm 2 is $O(|C^{G_{fi}^n}|)$. In part 1 of Algorithm 3, each local gateway calculates the total number of selected clients in its network and the maximum required time to complete the training and shares it with the global server (Steps 2, and 3) whereas, part 2 of Algorithm 3 sends the required number of client updates to the local gateways of filtered gateway set. If G_{fi}^n represents ith filtered gateway corresponding to ith data-type the complexity of both part 1 and part 2 is $O(|G_{fi}^n|)$. Hence, the total complexity of the proposed FedDCS in one FL round can be calculated as: Complexity (Algorithm 1) + Complexity (Algorithm 2) + Complexity (Algorithm 3) = $O(|G|) + O(|C^{G_{fi}^n}|) + O(|G_{fi}^n|)$. Thus, it can be concluded that FedAVG observes constant time complexity whereas FedCS and FedMCCS result in quadratic time complexities as compared to linear time complexity observed in the proposed FedDCS.

7. Conclusion and future work

A novel framework for distributed cross-device FL client selection known as FedDCS, addressing the resource-constrained IoT devices as FLClients is proposed in this paper. FedDCS guarantees that selected clients do not drop out in between FL training through localized resource monitoring enabled by edge servers. The performance of FedDCS is compared with three baseline approaches FedAVG, FedCS, and FedMCCS. The results obtained show that FedDCS outperforms all of them in terms of minimizing client drop-outs, time to reach desired accuracy, model convergence time and the number of message exchanges during the FL training process in both IID/Non-IID settings of FL.

Finally, future directions may include deploying mechanisms at local gateways to identify the unnecessary/redundant updates shared by the clients so that the communication cost can be reduced. In addition, resource-sharing by the neighbouring clients can be explored to execute resource-demanding model training tasks.

CRediT authorship contribution statement

Monalisa Panigrahi: Conceptualization, Methodology, Software, Investigation, Writing – original draft. **Sourabh Bharti:** Supervision, Methodology, Validation, Writing – review & editing. **Arun Sharma:** Supervision, Writing – review.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] P. Kairouz, H.B. McMahan, B. Avent, A. Bellet, M. Bennis, A.N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al., Advances and open problems in federated learning, Found. Trends Mach. Learn. 14 (2021) 1–210, now Publishers.

[2] T. Nishio, R. Yonetani, Client selection for federated learning with heterogeneous resources in mobile edge, in: IEEE International Conference on Communications, ICC, 2019, pp. 1–7.

[3] S. AbdulRahman, H. Tout, A. Mourad, C. Talhi, Fedmccs: multicriteria client selection model for optimal IoT federated learning, IEEE Internet Things J. 8 (2021) 4723–4735.

[4] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: 20th International Conference on Artificial Intelligence and Statistics, AISTATS, 2017, pp. 1273–1282.

[5] S. Bharti, A. Mcgibney, Privacy-aware resource sharing in cross-device federated model training for collaborative predictive maintenance, IEEE Access 9 (2021) 120367–120379.

[6] L. Liu, J. Zhang, S. Song, K.B. Letaief, Client-edge-cloud hierarchical federated learning, in: IEEE International Conference on Communications, ICC, 2020, pp. 1–6.

[7] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-iid data, 2018, arXiv preprint arXiv:1806.00582.

[8] J. Kang, Z. Xiong, D. Niyato, H. Yu, Y.-C. Liang, D.I. Kim, Incentive design for efficient federated learning in mobile networks: A contract theory approach, in: VTS Asia Pacific Wireless Communications Symposium, APWCS, 2019, pp. 1–5.
[9] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, Y. Cheng, Tifl: A tier-based federated learning system, in: Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing, 2020, pp. 125–136.
[10] N. Yoshida, T. Nishio, M. Morikura, K. Yamamoto, R. Yonetani, Hybrid-FL for wireless networks: Cooperative learning mechanism using non-IID data, in: IEEE International Conference on Communications, ICC, 2020, pp. 1–7.
[11] J. Song, M. Kountouris, Wireless distributed edge learning: How many edge devices do we need? IEEE J. Sel. Areas Commun. (2020).
[12] J. Jeon, S. Park, M. Choi, J. Kim, Y.-B. Kwon, S. Cho, Optimal user selection for high-performance and stabilized energy-efficient federated learning platforms, Electronics 9 (2020) 1359.
[13] J. Xu, H. Wang, Client selection and bandwidth allocation in wireless federated learning networks: A long-term perspective, IEEE Trans. Wireless Commun. 20 (2020) 1188–1200.
[14] W. Zhang, X. Wang, P. Zhou, W. Wu, X. Zhang, Client selection for federated learning with non-IID data in mobile edge computing, IEEE Access 9 (2021) 24462–24474.
[15] A.A. Abdellatif, N. Mhaisen, A. Mohamed, A. Erbad, M. Guizani, Z. Dawy, W. Nasreddine, Communication-efficient hierarchical federated learning for IoT heterogeneous systems with imbalanced data, Future Gener. Comput. Syst. 128 (2022) 406–419.
[16] X. Wang, Y. Wang, Asynchronous hierarchical federated learning, 2022, arXiv preprint arXiv:2206.00054.
[17] C. Liu, T.J. Chua, J. Zhao, Time minimization in hierarchical federated learning, 2022, arXiv preprint arXiv:2210.04689.
[18] J. Kang, Z. Xiong, D. Niyato, S. Xie, J. Zhang, Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory, IEEE Internet Things J. 6 (2019) 10700–10714.
[19] D. Ye, R. Yu, M. Pan, Z. Han, Federated learning in vehicular edge computing: A selective model aggregation approach, IEEE Access 8 (2020) 23920–23935.
[20] J. Kang, Z. Xiong, D. Niyato, Y. Zou, Y. Zhang, M. Guizani, Reliable federated learning for mobile networks, IEEE Wirel. Commun. 27 (2020) 72–80.
[21] Y. Wang, B. Kantarci, A novel reputation-aware client selection scheme for federated learning within mobile environments, in: 25th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks, CAMAD, 2020, pp. 1–6.
[22] Y. Sarikaya, O. Ercetin, Motivating workers in federated learning: A stackelberg game perspective, IEEE Netw. Lett. 2 (2019) 23–27.
[23] L.U. Khan, S.R. Pandey, N.H. Tran, W. Saad, Z. Han, M.N. Nguyen, C.S. Hong, Federated learning for edge networks: Resource optimization and incentive mechanism, IEEE Commun. Mag. 58 (2020) 88–93.
[24] Y. Zhan, P. Li, Z. Qu, D. Zeng, S. Guo, A learning-based incentive mechanism for federated learning, IEEE Internet Things J. 7 (2020) 6360–6368.
[25] N. Mhaisen, A.A. Abdellatif, A. Mohamed, A. Erbad, M. Guizani, Optimal user-edge assignment in hierarchical federated learning based on statistical properties and network topology constraints, IEEE Trans. Netw. Sci. Eng. 9 (2021) 55–66.
[26] V.L. Parsons, Stratified Sampling, Wiley, 2014, pp. 1–11, StatsRef: Statistics Reference Online.
[27] J.A. Alzubi, Bipolar fully recurrent deep structured neural learning based attack detection for securing industrial sensor networks, Trans. Emerg. Telecommun. Technol. 32 (2021) 4069.
[28] Q. Gao, K.J. Blow, D.J. Holding, I.W. Marshall, X. Peng, Radio range adjustment for energy efficient wireless sensor networks, Ad Hoc Netw. 4 (2006) 75–82.
[29] T. Ryffel, A. Trask, M. Dahl, B. Wagner, J. Mancuso, D. Rueckert, J. Passerat-Palmbach, A generic framework for privacy preserving deep learning, 2018, arXiv preprint arXiv:1811.04017.
[30] L. Deng, The mnist database of handwritten digit images for machine learning research [best of the web], IEEE Signal Process. Mag. 29 (2012) 141–142.
[31] H. Xiao, K. Rasul, R. Vollgraf, Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, 2017, arXiv preprint arXiv:1708.07747.
[32] M. Panigrahi, S. Bharti, A. Sharma, An exhaustive investigation on resource-aware client selection mechanisms for cross-device federated learning, in: Proceedings of the 2022 Fourteenth International Conference on Contemporary Computing, 2022, pp. 67–73.



Monalisa Panigrahi received an M.Tech degree in information technology from the Indian Institute of Information Technology, Allahabad, India. She is currently working towards a Ph.D. degree in information technology, at Indira Gandhi Delhi Technical University for Women (IGDTUW), Delhi, India. Her research interests include federated learning, the Internet of Things (IoT) and wireless sensor network.



Sourabh Bharti (Member, IEEE) received a Ph.D. degree in information technology from the Indian Institute of Information Technology and Management, Gwalior, India. He is currently a Marie Skłodowska-Curie Research Fellow with the CONFIRM Centre for Smart Manufacturing, Ireland and is based at Nimbus Research Centre, Munster Technological University, Cork, Ireland. His research interests include AI Applications, Industry 4.0, and the Internet of Things (IoT).



Arun Sharma (Senior Member, IEEE) is a Professor in the Department of Artificial Intelligence and Data Science, Indira Gandhi Delhi Technical University for Women (IGDTUW), Delhi, India. He received a Ph.D. from Thapar University in 2009. He has a teaching experience of more than 25 years. His research interests include Machine Learning, Data Science and Software Engineering. He is also working as Managing Director - IGDTUW Anveshan Foundation (Incubation Centre recognized as NIDHI TBI by DST, India).